

Public comments 2000-2025: text mining, EDA, and topic models

Table of contents

Simulated data (DELETE THIS SECTION AT WORK)	3
Corpus validation (KEEP THIS AT WORK)	4
Newline structure diagnostic	5
Paragraph atomization	10
Exploratory analysis	12
Document-term matrix	17
LDA comparator	18
STM: choosing K with a held-out grid	19
Final STM	22
Topic ranking by expected prevalence	22
Representative paragraphs	25
LLM topic-label drafting (optional, one call per topic)	26
Prevalence over time	29
Handoff to Script 2	30

What it does: defines tx, the single list every later chunk reads its settings from, attaches packages (tidyverse deliberately last so dplyr verbs win any masking contest), and defines the two check-point helpers. Why one config list: every analyst decision (splitter, thresholds, K, topic selection, LLM flags) is a line here, so adapting the script never means editing analysis code. Why checkpoints: `cache_rds()/cache_parquet()` compute an expression once, save it under `outputs/`, and reload it on every later render; the cost is that caches track files, not code, so anything edited upstream of a checkpoint requires deleting that file (or all of `outputs/`) once.

```
# -----  
# CONFIGURATION (the only block you edit)  
# -----  
tx <- list(  
  seed           = 2026L,  
  out_dir        = "outputs",  
  # paragraph atomization  
  para_split     = "\\r?\\n+",      # newline-run splitter (CRLF or LF, single  
                                   # or double breaks). PRODUCTION: if the  
                                   # newline diagnostic shows single newlines  
                                   # are soft wraps INSIDE paragraphs, change  
                                   # to "\\r?\\n{2,}" (blank-line separated)  
  para_min_tokens = 15L,          # body fragments shorter than this merge  
                                   # into the PRECEDING paragraph  
)
```

```

heading_max_tokens = 12L,      # blocks this short WITHOUT sentence-ending
                                # punctuation are treated as headings and
                                # merged FORWARD into the paragraph they
                                # introduce (see the atomization section)

# searchK
run_searchK      = TRUE,      # expensive; set FALSE after choosing K
K_grid           = seq(4L, 80L, by = 3L),
heldout_prop     = 0.10,      # share of docs with held-out words
# final models
K                = 15L,      # PLACEHOLDER for the simulation; set from
                                # the searchK diagnostics on real data
spline_df        = 5L,      # Year spline df for topical prevalence
# parallelism
workers          = 6L,
# LLM topic-label drafting (one call per topic; drafts only, analyst verifies)
run_llm_labels   = FALSE,      # TRUE once the API key is in .Renvirom
llm_model        = "openrouter/meta-llama/llama-4-maverick",
                                # WORK SWAP: this ONE line. Any string not
                                # starting with "openrouter/" is routed to
                                # the OpenAI-compatible work endpoint (see
                                # make_label_chat in the labeling section).

# topic selection for Script 2 (set AFTER naming topics from the FREX sheet)
selected_topics = NULL      # e.g. c(3L, 7L, 11L)
)

suppressPackageStartupMessages({
  library(arrow)      # parquet outputs
  library(quanteda)   # dfm construction and conversion to stm format
  library(topicmodels) # LDA comparator
  library(stm)        # structural topic model
  library(tidytext)   # tidy tokenization and tidiers for LDA
  library(furrr)      # parallel map for the searchK grid
  library(tidyverse)  # loaded LAST so dplyr verbs win
})
theme_set(theme_minimal(base_size = 12))
set.seed(tx$seed)
dir.create(tx$out_dir, showWarnings = FALSE)

# Checkpoint helpers: compute once, then reload. Delete the file to re-fit.
# IMPORTANT: caches track FILES, not code. After editing anything upstream of a
# checkpoint (the simulator, the atomizer, the dfm), delete that checkpoint file,
# or delete the whole outputs/ folder to rebuild everything from scratch.
cache_rds <- function(path, expr) {
  if (file.exists(path)) return(readRDS(path))
  x <- force(expr); saveRDS(x, path); x
}
cache_parquet <- function(path, expr) {
  if (file.exists(path)) return(arrow::read_parquet(path))

```

```
x <- force(expr); arrow::write_parquet(x, path); x
}
p_out <- function(f) file.path(tx$out_dir, f)
```

Simulated data (DELETE THIS SECTION AT WORK)

PRODUCTION: delete this whole section and replace it with the one-line read, `corpus_raw <- arrow::read_parquet("D:/path/to/comments.parquet")` (absolute path; here() anchors to the OneDrive root, not the repo). Nothing after the validation gate knows or cares which source filled `corpus_raw`.

The block below fabricates ~1,200 multi-paragraph comments over 2000-2025 so the pipeline runs end to end. Each simulated comment mixes 2-4 of six themes across its paragraphs, mimicking the real problem: long comments spanning multiple topics. At work, delete everything in this section and read your real table into `corpus_raw` with columns `atom_id`, `Year`, `body_english`.

```
theme_vocab <- list(
  health      = c("vaccine", "hospital", "clinic", "disease", "outbreak",
    "screening",
    "nurses", "patients", "epidemic", "prevention", "mortality",
    "insurance",
    "treatment", "wellness", "sanitation", "immunization", "pandemic",
    "care"),
  economy     = c("jobs", "wages", "inflation", "budget", "taxes", "employment",
    "trade",
    "industry", "investment", "growth", "deficit", "subsidies",
    "tariffs",
    "market", "workers", "manufacturing", "exports", "recession"),
  security    = c("police", "crime", "border", "defense", "surveillance",
    "terrorism",
    "safety", "enforcement", "military", "threat", "protection",
    "patrol",
    "intelligence", "violence", "weapons", "emergency", "gangs",
    "courts"),
  environment = c("climate", "emissions", "pollution", "wildlife", "rivers",
    "forests",
    "renewable", "solar", "drought", "flooding", "conservation",
    "recycling",
    "pesticides", "wetlands", "carbon", "warming", "habitat",
    "cleanup"),
  education   = c("schools", "teachers", "students", "curriculum", "tuition",
    "literacy",
    "classrooms", "funding", "graduation", "testing", "scholarships",
    "universities", "training", "libraries", "enrollment", "tutoring"),
  housing     = c("rent", "landlords", "eviction", "zoning", "homeless", "mortgage",
    "affordable", "construction", "tenants", "shelters", "development",
    "neighborhood", "property", "vacancy", "permits", "gentrification")
```

```

)
filler <- c("the", "a", "of", "and", "to", "in", "we", "our", "is", "that",
"for",
          "should", "must", "will", "not", "more", "less", "very", "public",
          "community", "policy", "government", "people", "city", "council",
          "issue", "concern", "support", "oppose", "believe", "think", "need")

# Era-specific vocabulary so the corpus drifts over time the way real comments
# do; without it every term appears in every era, all idf terms are log(1) = 0,
# and the era TF-IDF view is degenerate by construction.
era_vocab <- list(
  c("y2k", "dialup", "anthrax", "millennium", "modem"),          # 2000-2004
  c("foreclosure", "stimulus", "hurricane", "bailout", "blog"), # 2005-2009
  c("austerity", "fracking", "broadband", "drone", "hashtag"),  # 2010-2014
  c("opioid", "wildfire", "streaming", "rideshare", "vaping"),  # 2015-2019
  c("lockdown", "telework", "quarantine", "misinformation", "chatbot") #
  2020-2025
)
era_of <- function(year) findInterval(year, c(2000, 2005, 2010, 2015, 2020))

make_paragraph <- function(theme, year) {
  n <- sample(40:120, 1)
  words <- sample(c(theme_vocab[[theme]], filler), n, replace = TRUE,
                  prob = c(rep(3, length(theme_vocab[[theme]])), rep(1,
length(filler))))
  extra <- sample(era_vocab[[era_of(year)]], sample(2:5, 1), replace = TRUE)
  str_c(str_c(sample(c(words, extra)), collapse = " "), ".")
}
make_comment <- function(id) {
  yr <- sample(2000:2025, 1, prob = 0.5 + seq(0, 1, length.out = 26))
  themes_i <- sample(names(theme_vocab), sample(2:4, 1))
  n_para <- sample(2:8, 1)
  paras <- map_chr(sample(themes_i, n_para, replace = TRUE), make_paragraph,
year = yr)
  tibble(atom_id = sprintf("A%05d", id),
         Year = yr,
         body_english = str_c(paras, collapse = "\n\n"))
}
corpus_raw <- map_dfr(seq_len(1200), make_comment)

```

Corpus validation (KEEP THIS AT WORK)

This gate runs on whatever `corpus_raw` holds, simulated or real. Every check names its failure, so a bad real-data read stops here with a diagnosis instead of surfacing as an opaque error downstream. Reading the real table is one line before this chunk, e.g. `corpus_raw <- arrow::read_parquet("path/to/comments.parquet")` or `readr::read_csv(..., show_col_types = FALSE)`; if the source columns have different names,

rename them here once (`dplyr::rename(atom_id = ..., Year = ..., body_english = ...)`) so nothing downstream changes.

```
required_cols <- c("atom_id", "Year", "body_english")
missing_cols <- setdiff(required_cols, names(corpus_raw))
if (length(missing_cols) > 0)
  stop("corpus_raw is missing column(s): ", str_c(missing_cols, collapse = ",
"))

stopifnot(
  "atom_id must be non-missing" = !anyNA(corpus_raw$atom_id),
  "atom_id must be unique (one row per comment)" =
    !anyDuplicated(corpus_raw$atom_id),
  "Year must be integer-valued" =
    is.numeric(corpus_raw$Year) && all(corpus_raw$Year ==
as.integer(corpus_raw$Year), na.rm = TRUE),
  "body_english must be character" = is.character(corpus_raw$body_english)
)

# Non-fatal quality report: rows dropped here are removed with a count, not
# silently. NA years cannot enter the STM prevalence spline; empty bodies
# cannot atomize.
n_before <- nrow(corpus_raw)
corpus_raw <- corpus_raw |>
  dplyr::filter(!is.na(Year), !is.na(body_english), str_squish(body_english) != "")
cat("Validation: kept", nrow(corpus_raw), "of", n_before, "comments;",
    "Year range", min(corpus_raw$Year), "-", max(corpus_raw$Year), "\n")
```

```
Validation: kept 1200 of 1200 comments; Year range 2000 - 2025
```

```
if (any(corpus_raw$Year < 2000 | corpus_raw$Year > 2025))
  warning("Years outside 2000-2025 present; confirm this is expected before ",
    "interpreting the Year spline.")
```

Newline structure diagnostic

What it does: counts, per comment, how many line breaks the text carries and how many blank-line separations, then prints three comments with every break made visible as `<NL>`. Why: the atomizer can only split on whatever the source uses to mark paragraphs, and Excel-origin text (in-cell Alt+Enter breaks survive `readxl` and `parquet` as `\n`) can carry three different conventions that need three different responses. Decision rules, applied once on the real corpus:

1. `zero_newlines` small and the rest have breaks: keep the default `tx$para_split`. Break-free comments simply stay single paragraphs, which is correct.

2. `has_blank_lines` dominates AND the `<NL>` sample shows single breaks used as soft wraps inside paragraphs: paragraphs are blank-line separated; set `tx$para_split = "\\r?\\n{2,}"` in the config. Only a human reading the sample can tell soft wraps from true breaks, which is why the counts alone do not decide.
3. `zero_newlines` is most of the corpus: there is nothing to split on and the fallback is sentence-boundary chunking to a token budget, a design change to the measurement unit that must be decided deliberately, not patched in.

```
newline_report <- corpus_raw |>
  mutate(body_lf = str_replace_all(body_english, "\\r\\n", "\\n"),
         n_breaks = str_count(body_lf, "\\n"),
         n_blank = str_count(body_lf, "\\n{2,}")) |>
  summarise(comments = n(),
            zero_newlines = sum(n_breaks == 0),
            single_only = sum(n_breaks > 0 & n_blank == 0),
            has_blank_lines = sum(n_blank > 0),
            median_breaks = median(n_breaks))
knitr::kable(newline_report,
              caption = "Line-break conventions in the corpus; apply the decision
rules in the prose above before the long render.")
```

comments	zero_newlines	single_only	has_blank_lines	median_breaks
1200	0	0	1200	8

Table 1: Line-break conventions in the corpus; apply the decision rules in the prose above before the long render.

```
corpus_raw |>
  dplyr::slice_sample(n = 3) |>
  pull(body_english) |>
  str_replace_all("\\r?\\n", "<NL>\\n") |>
  cat(sep = "\\n\\n=== NEXT COMMENT ===\\n\\n")
```

violence and a issue issue crime less policy weapons enforcement weapons will
surveillance protection must safety violence need surveillance and chatbot need
support terrorism support issue surveillance chatbot quarantine support border
military border surveillance threat we must courts crime threat weapons is in
patrol think should think police protection protection patrol.<NL>

<NL>

believe recycling drought emissions pesticides renewable pollution recycling
our drought drought concern conservation pollution lockdown very public council
pesticides telework renewable solar wildlife renewable rivers wetlands not
more flooding in misinformation pesticides will not a to wildlife pollution
must flooding renewable we emissions cleanup public pesticides forests cleanup
community conservation must misinformation solar conservation support pesticides

habitat carbon is that pesticides climate oppose pollution misinformation climate forests wildlife.<NL>

<NL>

immunization quarantine think sanitation vaccine for community and immunization city screening care immunization city care mortality treatment sanitation need telework mortality sanitation is community clinic mortality clinic wellness very outbreak hospital care epidemic clinic clinic mortality wellness must mortality council very support nurses support insurance treatment for city vaccine disease treatment care that very sanitation and vaccine clinic will that is hospital policy care and treatment screening outbreak nurses need treatment treatment immunization think pandemic government need disease not people wellness and mortality pandemic patients chatbot mortality epidemic support patients insurance issue patients mortality patients we very care epidemic.<NL>

<NL>

courts enforcement military defense crime protection intelligence should oppose need terrorism terrorism emergency military policy we emergency terrorism we for surveillance city patrol misinformation military need defense not lockdown patrol for weapons very we courts intelligence patrol weapons support patrol gangs enforcement border emergency gangs that council protection and protection crime lockdown will in terrorism surveillance must courts community gangs intelligence need intelligence courts in emergency border oppose police not council city military that threat threat telework defense for gangs crime police terrorism patrol the the weapons defense and defense.<NL>

<NL>

terrorism military chatbot think weapons intelligence patrol public quarantine misinformation should military emergency patrol more courts believe must we quarantine support military for intelligence city surveillance is safety of enforcement intelligence concern border surveillance weapons emergency weapons enforcement oppose people policy public military to protection surveillance threat people enforcement military is our should border enforcement patrol and police more surveillance to.<NL>

<NL>

border enforcement will crime should police intelligence border crime oppose enforcement terrorism safety less safety violence a defense intelligence safety emergency is we courts intelligence for of not our misinformation for patrol terrorism to will police quarantine safety weapons terrorism emergency emergency surveillance weapons police the police misinformation border patrol military weapons violence enforcement enforcement border issue defense intelligence police city telework safety courts policy border very more telework not terrorism support need.<NL>

<NL>

will wetlands warming pesticides policy very community pesticides think climate cleanup chatbot warming and renewable that pesticides telework solar forests oppose climate council believe more warming wetlands warming climate people wetlands wildlife cleanup solar government conservation we pesticides forests wetlands conservation habitat think very council wildlife flooding pesticides misinformation renewable a flooding our forests more cleanup drought warming wildlife council cleanup habitat will renewable people wetlands the support

government to we.<NL>

<NL>

enforcement police weapons crime border military support protection police
enforcement the patrol less military border misinformation think think
misinformation violence safety military in military city gangs a patrol need
the courts violence to courts a police misinformation protection is protection
protection enforcement believe enforcement intelligence believe defense patrol
safety believe military enforcement public quarantine terrorism weapons police
surveillance military intelligence violence public patrol public need must threat
for should should gangs defense think violence border surveillance crime a
government patrol council courts threat in weapons police safety council defense
very safety emergency.

==== NEXT COMMENT ====

habitat concern misinformation lockdown climate quarantine solar pollution
our pesticides less government public cleanup drought solar solar emissions
pesticides should must believe drought conservation climate cleanup
misinformation rivers less solar recycling pesticides pollution the government
less rivers that climate recycling emissions drought concern recycling emissions
habitat emissions less in emissions flooding emissions cleanup flooding
quarantine government should warming very climate pollution solar wildlife
renewable need carbon flooding drought flooding believe drought pesticides
recycling carbon for warming drought wildlife flooding carbon recycling warming
people cleanup solar forests rivers warming policy oppose climate recycling solar
in emissions carbon public flooding rivers wildlife.<NL>

<NL>

renewable wetlands misinformation lockdown solar emissions pollution warming
conservation flooding climate concern flooding should conservation issue
pesticides wildlife pollution climate forests pesticides will cleanup emissions
need warming city in climate pollution council warming support conservation
warming wildlife drought forests carbon carbon climate climate conservation
believe believe a flooding the and forests very the emissions that rivers
public cleanup wetlands should our wildlife cleanup wetlands carbon renewable
carbon.<NL>

<NL>

need tutoring is scholarships chatbot policy teachers enrollment concern testing
funding graduation funding a libraries libraries libraries oppose teachers
lockdown schools policy libraries lockdown issue less training community people
literacy lockdown universities support more funding tutoring tuition schools
very will schools teachers classrooms city scholarships schools teachers should
must graduation classrooms enrollment scholarships funding concern testing issue
a testing tuition graduation teachers that government libraries testing for less
and graduation more not misinformation our graduation students is funding very
a.<NL>

<NL>

support training and schools council literacy that will testing community
tuition scholarships tuition must is chatbot tutoring tutoring less teachers in

graduation classrooms classrooms is scholarships testing schools teachers will training teachers tutoring testing universities testing enrollment teachers city scholarships our more students teachers quarantine scholarships scholarships of teachers schools literacy students quarantine oppose enrollment libraries universities will literacy community the tuition a and testing testing universities training to funding oppose concern very for issue testing training and.<NL>

<NL>

training scholarships is quarantine teachers teachers enrollment a schools lockdown libraries classrooms students schools should for graduation teachers tuition tuition libraries not should lockdown curriculum schools will universities testing tuition policy not more schools in need enrollment training policy testing for public very testing testing support students people.<NL>

<NL>

that classrooms tuition schools curriculum tutoring libraries enrollment tuition funding funding schools tuition lockdown libraries tutoring curriculum enrollment concern of curriculum tuition libraries students tutoring students support misinformation teachers training libraries libraries for tutoring believe universities should funding teachers schools scholarships very funding students scholarships support literacy think council testing funding classrooms funding less our enrollment we testing scholarships libraries a oppose curriculum universities community think universities graduation.<NL>

<NL>

intelligence police military protection patrol border policy oppose lockdown policy military enforcement less police patrol emergency will support we border police in and violence surveillance intelligence military enforcement think enforcement defense safety courts police to public surveillance crime weapons protection gangs police border less courts intelligence safety the violence that issue violence need emergency we intelligence defense protection emergency council that enforcement protection lockdown patrol policy surveillance for surveillance terrorism emergency to that protection policy we emergency in weapons believe community the and is military we the think emergency protection intelligence border community police enforcement enforcement military safety crime weapons courts emergency surveillance crime for defense terrorism crime safety.

==== NEXT COMMENT ====

city clinic oppose y2k disease not treatment support hospital mortality hospital screening city treatment city to insurance prevention outbreak to nurses patients issue concern in insurance is y2k insurance wellness wellness wellness prevention is pandemic sanitation council that to treatment insurance hospital screening patients prevention patients community clinic disease prevention hospital believe sanitation our city issue hospital not epidemic need that government anthrax vaccine we screening.<NL>

<NL>

a that terrorism violence is enforcement threat crime police is weapons support that patrol weapons safety is that of emergency people violence border community

gangs crime violence believe safety police weapons oppose issue public less
dialup emergency dialup millennium will terrorism surveillance and military
enforcement oppose crime border terrorism intelligence a defense dialup weapons
in for defense terrorism police terrorism terrorism violence patrol very issue
border a patrol patrol to is threat protection the patrol border threat weapons
issue emergency oppose.<NL>

<NL>

patients insurance will our patients public wellness vaccine people prevention
nurses in concern hospital treatment epidemic for insurance patients support
people treatment treatment care patients must y2k hospital wellness screening
very care a prevention hospital issue that outbreak oppose should mortality care
clinic very modem think prevention epidemic very prevention our will insurance
care anthrax for.<NL>

<NL>

curriculum tutoring teachers enrollment must training curriculum literacy
schools for millennium tutoring support enrollment graduation public classrooms
universities oppose tuition testing oppose concern universities very teachers
training enrollment curriculum think funding enrollment tutoring classrooms
tutoring graduation scholarships of schools support issue anthrax believe will
and for literacy not must we schools council.<NL>

<NL>

care pandemic epidemic council nurses anthrax wellness concern vaccine insurance
screening city people treatment that council patients y2k more hospital clinic
care people and millennium immunization wellness millennium public treatment
pandemic think insurance council oppose government think vaccine epidemic council
nurses issue outbreak and millennium.<NL>

<NL>

training less government concern libraries testing tutoring training testing
students scholarships tutoring enrollment teachers less funding literacy
scholarships in will training schools universities in issue classrooms libraries
dialup enrollment training our believe government libraries scholarships
funding libraries the training curriculum community dialup city less government
city community enrollment graduation funding testing concern universities
universities funding literacy students must training training support support
concern tutoring issue teachers anthrax millennium tutoring tuition graduation
need city curriculum policy classrooms literacy and community support funding
council curriculum public of literacy policy training graduation should people
council literacy is tutoring testing funding schools.

Paragraph atomization

What it does: splits each comment on `tx$para_split`, then reassembles short blocks into full paragraphs with a **direction-aware** merge, and assigns each paragraph a stable id. Why the paragraph: the estimand downstream is stance on a topic, but a comment can span several topics; LDA and STM define a “document” as whatever unit the document-term matrix is built over, and here that unit is the paragraph. Each paragraph keeps its parent `atom_id` and `Year`, and gains a `para_id` nested within the comment.

The merge runs in two passes because short blocks come in two kinds that belong in opposite directions:

- **Headings and subtitles merge FORWARD.** A block of at most 12 tokens with no sentence-ending punctuation (so “Housing Concerns”, “1. Introduction”, “Background:”) is a heading: it announces the content that follows, so its words are attached to the *next* body paragraph. Merging it backward, as a naive short-fragment rule would, contaminates the *previous* paragraph’s topic signal with the next section’s vocabulary, exactly the wrong assignment for a topic model. A heading with nothing after it falls back to the last body paragraph rather than being dropped.
- **Body fragments merge BACKWARD.** A block under 15 tokens that does end like a sentence (“No.”, “I strongly oppose this.”) is a continuation or an emphatic coda of what precedes it, so it joins the preceding paragraph, the original rule. Salutations and signature lines are absorbed the same way instead of surviving as junk paragraphs.

The heading heuristic is deliberately simple (length plus terminal punctuation) so its behavior is auditable; after the first real render, check it two ways: `quantile(paragraphs$n_tokens)` for the length distribution, and a read of a few dozen paragraphs for headings glued to the wrong side. Both `tx$heading_max_tokens` and `tx$para_min_tokens` are config knobs.

```
count_tokens <- function(x) str_count(x, "\\S+")

paragraphs <- cache_parquet(p_out("paragraphs.parquet"), {
  # PASS 1 (forward): attach headings to the paragraph they introduce.
  # rev(cumsum(rev(!heading))) gives every body block a distinct id that
  # DECREASES down the document, and gives a heading the same id as the next
  # body block, so grouping on it merges the heading forward. Trailing
  # headings get id 0 and pmax(1L) folds them into the last body block.
  blocks <- corpus_raw |>
    mutate(para_raw = str_split(body_english, tx$para_split)) |>
    dplyr::select(atom_id, Year, para_raw) |>
    unnest(para_raw) |>
    mutate(para_raw = str_squish(para_raw)) |>
    dplyr::filter(para_raw != "") |>
    group_by(atom_id, Year) |>
    mutate(heading = count_tokens(para_raw) <= tx$heading_max_tokens &
           !str_detect(para_raw, "[.!?]['\"]\\S*"),
           fwd_block = rev(cumsum(rev(!heading))) |> pmax(1L)) |>
    group_by(atom_id, Year, fwd_block) |>
    summarise(para_raw = str_c(para_raw, collapse = " "), .groups = "drop") |>
    dplyr::arrange(atom_id, Year, dplyr::desc(fwd_block)) # restore doc order

  # PASS 2 (backward): merge remaining short body fragments into the
  # preceding paragraph; a fragment gets the running index of the last
  # full-size paragraph before it.
  blocks |>
    group_by(atom_id, Year) |>
    mutate(full = count_tokens(para_raw) >= tx$para_min_tokens,
```

```

      block = cummax(cumsum(full) * full) |> pmax(1L)) |>
group_by(atom_id, Year, block) |>
summarise(text = str_c(para_raw, collapse = " "), .groups = "drop_last") |>
mutate(para_id = row_number(),
      para_uid = str_c(atom_id, "-", sprintf("%02d", para_id)),
      n_tokens = count_tokens(text)) |>
ungroup() |>
dplyr::filter(n_tokens >= tx$para_min_tokens) |>
dplyr::select(para_uid, atom_id, para_id, Year, n_tokens, text)
})
glimpse(paragraphs)

```

```

Rows: 5,975
Columns: 6
$ para_uid <chr> "A00001-01", "A00001-02", "A00001-03", "A00001-04", "A00001-0...
$ atom_id <chr> "A00001", "A00001", "A00001", "A00001", "A00001", "A00001", "...
$ para_id <int> 1, 2, 3, 4, 5, 6, 7, 8, 1, 2, 3, 4, 5, 6, 7, 1, 2, 3, 4, 5, 6...
$ Year <int> 2011, 2011, 2011, 2011, 2011, 2011, 2011, 2011, 2004, 2004, 2...
$ n_tokens <int> 60, 83, 82, 46, 45, 104, 87, 117, 94, 85, 69, 101, 47, 76, 10...
$ text <chr> "disease more very hospital care nurses treatment hospital sc...

```

The corpus atomizes to 5975 paragraphs from 1200 comments, a median of 5 paragraphs per comment.

Exploratory analysis

What it does: counts comments and paragraphs per year. Why: this is the denominator picture for everything time-related downstream; thin years mean the STM Year spline is interpolating there and single-year claims are weak, and a sudden paragraph-per-comment shift flags a change in how posts were written or ingested rather than in what they discuss.

```

paragraphs |>
group_by(Year) |>
summarise(paragraphs = n(), comments = n_distinct(atom_id), .groups = "drop")
|>
pivot_longer(c(comments, paragraphs), names_to = "unit", values_to = "n") |>
ggplot(aes(Year, n, fill = unit)) +
geom_col(position = "dodge") +
scale_fill_viridis_d(end = 0.7) +
labs(x = NULL, y = "count", fill = NULL,
      title = "Volume by year")

```

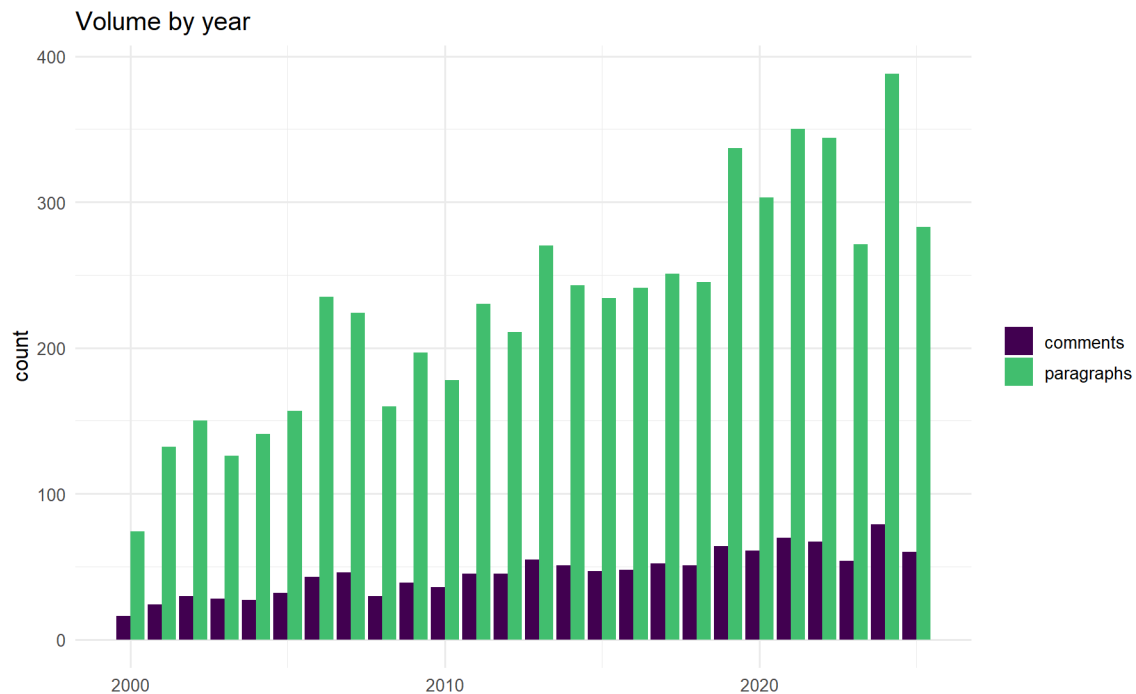


Figure 1: Corpus volume over time: comments and paragraphs per year.

Tokenization uses tidytext: one row per word per paragraph, lowercased, with stop words removed for the frequency views (topic models below get the same treatment). `custom_stops` is the domain stop list: it starts empty and is filled at work with corpus boilerplate that carries no topical signal but would otherwise dominate every topic (e.g. the agency’s own name, “comment”, “docket”, form-letter phrases). Fill it iteratively: render, read the top-terms plot and the topic word sheets, add offenders, delete outputs/, re-render.

```
# Domain stop words: terms so ubiquitous in THIS corpus that they behave like
# function words. Empty for the simulation; populate from the real top-terms
# plot and FREX sheets. Removing a word here removes it from the topic models
# too (they consume the same `tokens` table).
custom_stops <- tibble(word = character())

tokens <- paragraphs |>
  unnest_tokens(word, text) |>
  anti_join(stop_words, by = "word") |>
  anti_join(custom_stops, by = "word") |>
  dplyr::filter(str_detect(word, "[a-z]"))

top_terms <- tokens |> dplyr::count(word, sort = TRUE)
```

What it does: the raw frequency view after stop-word removal. Why: this is the fastest read on what dominates the corpus, and it is the primary hunting ground for `custom_stops`; a term at

the top that carries no topical meaning in this domain (the agency name, “comment”, form-letter phrases) belongs in the domain stop list before the topic models see it.

```
top_terms |>
  slice_head(n = 25) |>
  mutate(word = fct_reorder(word, n)) |>
  ggplot(aes(n, word, fill = n)) +
  geom_col(show.legend = FALSE) +
  scale_fill_viridis_c() +
  labs(x = "occurrences", y = NULL, title = "Top 25 terms")
```

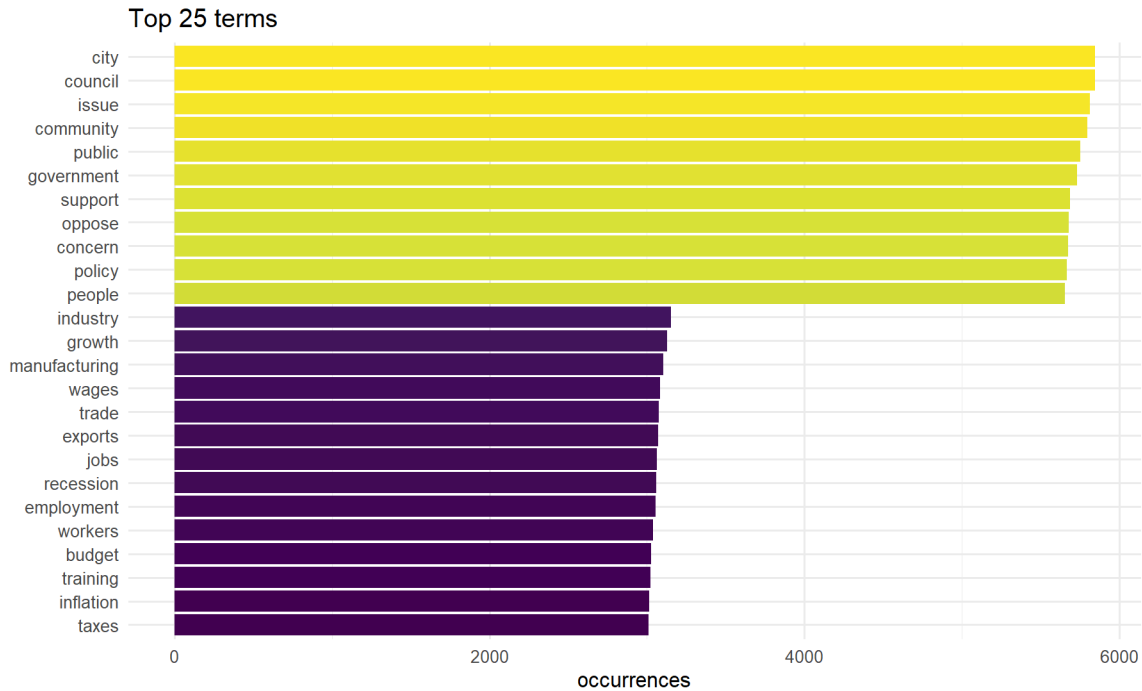


Figure 2: Most frequent terms after stop-word removal.

TF-IDF weighs a term by how concentrated it is in a subset of the corpus. For term t in document d ,

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{n_t},$$

where $\text{tf}(t, d)$ is the term’s share of document d ’s tokens, N is the number of documents, and n_t the number of documents containing t . Terms frequent everywhere score near zero; terms frequent in few places score high. Using the Year as the “document” surfaces era-specific vocabulary. Terms present in every era carry $\log(N/n_t) = \log 1 = 0$ and are filtered out: an empty or thin panel here means the vocabulary barely drifts across eras, which is itself a finding, not an error.

```

era_tfidf <- tokens |>
  mutate(era = str_c(Year %/% 5 * 5, "-", Year %/% 5 * 5 + 4)) |>
  dplyr::count(era, word) |>
  bind_tf_idf(word, era, n) |>
  dplyr::filter(tf_idf > 0) |> # terms in every era have idf = log(1) = 0 by
definition
  group_by(era) |>
  slice_max(tf_idf, n = 8, with_ties = FALSE) |>
  ungroup()

# An empty result means no term is era-distinctive (every term appears in every
# era). That is a statement about the corpus, so report it; do not crash on it.
if (nrow(era_tfidf) == 0) {
  cat("No era-distinctive terms: every term appears in every era, so all idf",
      "values are log(1) = 0. If this happens on cached simulated data, delete",
      "the outputs/ folder to rebuild the corpus.\n")
} else {
  p_era <- era_tfidf |>
    mutate(word = reorder_within(word, tf_idf, era)) |>
    ggplot(aes(tf_idf, word, fill = era)) +
    geom_col(show.legend = FALSE) +
    scale_y_reordered() +
    scale_fill_viridis_d() +
    facet_wrap(~era, scales = "free_y") +
    labs(x = "TF-IDF", y = NULL, title = "Era-distinctive vocabulary")
  print(p_era)
}

```

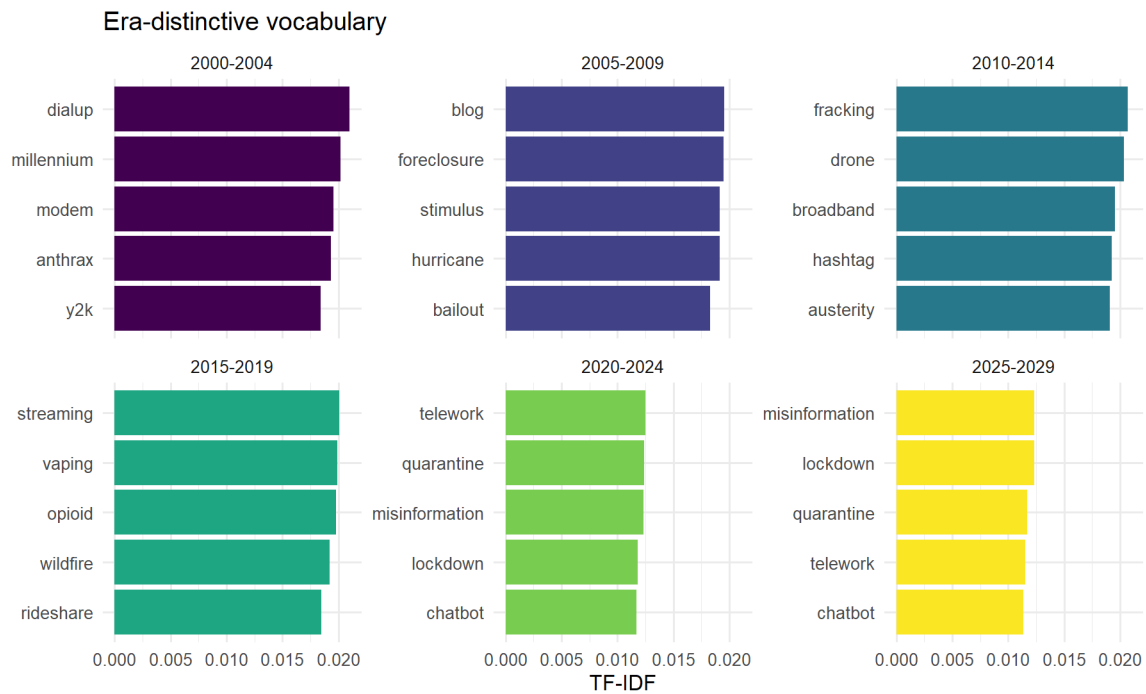


Figure 3: Highest TF-IDF terms by five-year era (Year as the document unit).

Lexicon sentiment gives a cheap corpus-level trend line before any modeling. It is a coarse instrument (word-match counts, no negation handling) and is used here for description only; the defensible sentiment measurement happens in Script 2.

```
tokens |>
  inner_join(get_sentiments("bing"), by = "word") |>
  dplyr::count(Year, sentiment) |>
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) |>
  mutate(share_positive = positive / (positive + negative)) |>
  ggplot(aes(Year, share_positive)) +
  geom_line(color = viridis::viridis(1, begin = 0.3), linewidth = 1) +
  geom_point(color = viridis::viridis(1, begin = 0.3)) +
  geom_hline(yintercept = 0.5, linetype = 2, color = "grey50") +
  labs(x = NULL, y = "share positive", title = "Lexicon sentiment over time")
```

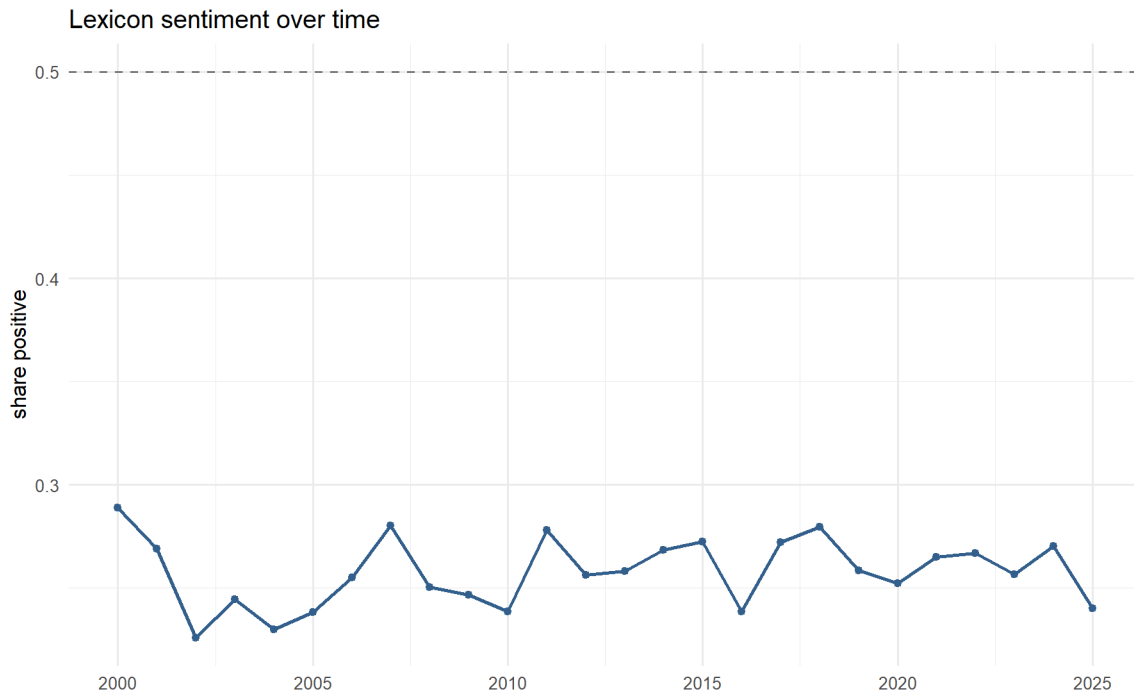



Figure 4: Bing lexicon sentiment by year: share of matched words that are positive.

Document-term matrix

Both topic models consume the same paragraph-by-term count matrix. Rare terms (fewer than 5 paragraphs) are dropped to stabilize estimation; the quanteda dfm converts directly to stm's input format, keeping para_uid alignment.

Adapting at work: `min_docfreq = 5` is calibrated to a corpus of a few thousand paragraphs. On the real corpus, check the vocabulary size after trimming (`ndoc(dfm_para)` and `nfeat(dfm_para)`); a vocabulary in the low thousands is comfortable for STM at this scale. If `nfeat` is much larger, raise the floor (or switch to a proportional floor, `dfm_trim(min_docfreq = 0.001, docfreq_type = "prop")`); if topics look starved of distinctive words, lower it. Also confirm no paragraph became empty after trimming: `quanteda::convert` drops empty documents, and if any are dropped the meta construction below still aligns because it is built from the surviving `rownames(dfm_para)`, but the paragraph counts reported earlier will no longer match the modeled corpus, which is worth a sentence in the write-up.

```
dfm_para <- tokens |>
  dplyr::count(para_uid, word) |>
  cast_dfm(para_uid, word, n) |>
  dfm_trim(min_docfreq = 5)

stm_input <- quanteda::convert(dfm_para, to = "stm")
# carry the covariate in dfm row order
meta <- tibble(para_uid = rownames(dfm_para)) |>
```

```
left_join(paragraphs |> dplyr::select(para_uid, atom_id, Year), by =
"para_uid")
stm_input$meta <- as.data.frame(meta)
```

LDA comparator

Latent Dirichlet Allocation is the baseline generative model: each document d has topic proportions $\theta_d \sim \text{Dirichlet}(\alpha)$, each topic k a term distribution $\beta_k \sim \text{Dirichlet}(\eta)$, and each token is drawn by picking a topic from θ_d then a word from that topic's β_k . It is fit here at the same K as the final STM as a sanity comparator: if STM's topics have no recognizable LDA counterparts, the STM structure (not the data) is doing the work, which is worth knowing.

```
lda_fit <- cache_rds(p_out("lda_fit.rds"), {
  topicmodels::LDA(
    quanteda::convert(dfm_para, to = "topicmodels"),
    k = tx$K, control = list(seed = tx$seed)
  )
})
```

What it does: plots each LDA topic's highest-probability words (beta). Why: this is the comparison sheet against the STM topics later; recognizable one-to-one counterparts mean the topic structure is in the data, while STM topics with no LDA analogue are being shaped by the prevalence model and deserve extra scrutiny before selection.

```
tidy(lda_fit, matrix = "beta") |>
  group_by(topic) |>
  slice_max(beta, n = 7, with_ties = FALSE) |>
  ungroup() |>
  mutate(term = reorder_within(term, beta, topic)) |>
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  scale_y_reordered() +
  scale_fill_viridis_d() +
  facet_wrap(~topic, scales = "free") +
  labs(x = expression(beta), y = NULL, title = str_c("LDA top terms, K = ", tx$K))
```

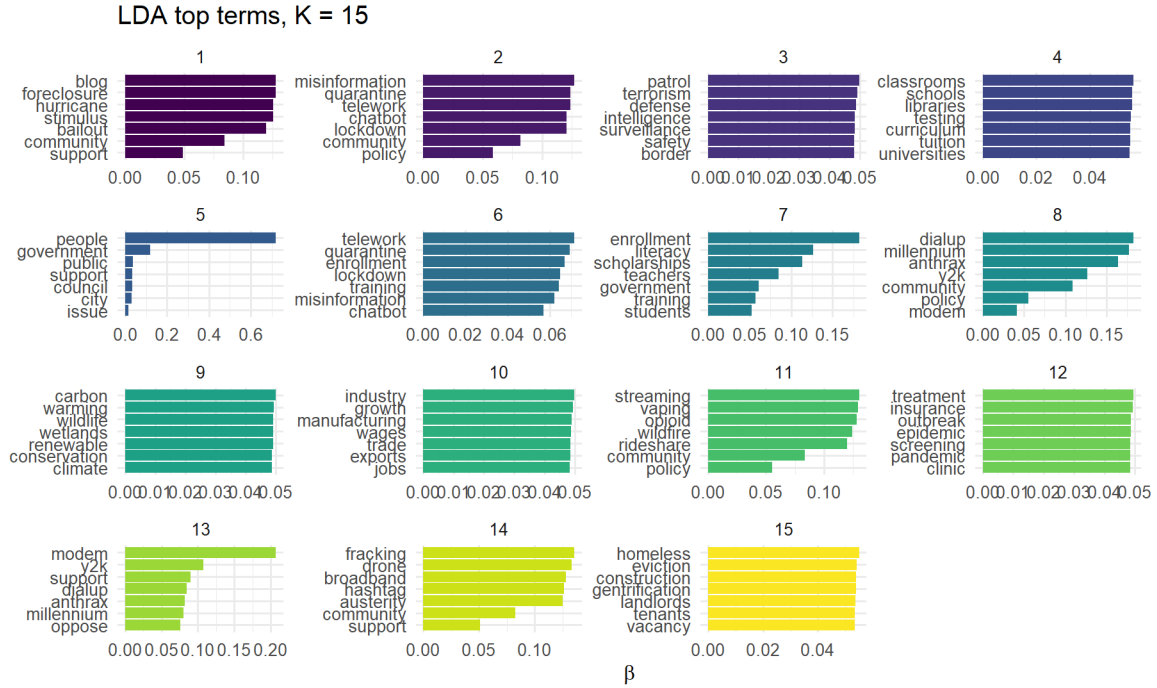


Figure 5: LDA: top terms per topic (beta, the topic-word distribution).

STM: choosing K with a held-out grid

The structural topic model replaces LDA's global θ prior with a regression: topical prevalence depends on covariates, here a smooth function of Year, so topic attention can drift over 2000-2025 and that drift is estimated rather than averaged away.

searchK's internal parallelism forks, which is unavailable on Windows, so the grid is parallelized explicitly with furr: hold out a random 10% of word tokens, fit each K on the remainder, and score four diagnostics.

Adapting at work: this is the long chunk. The grid has 26 values of K and each fit's cost grows with K , the number of paragraphs, and the vocabulary, so budget hours, not minutes, for the first real-data run. Each furr worker holds its own copy of the documents, so `tx$workers` is bounded by RAM as much as by cores; if the machine swaps, lower it. The run is checkpointed to `searchK_metrics.parquet`, so a crash or an aborted render costs only the incomplete grid, and once `tx$K` is chosen, set `tx$run_searchK = FALSE` so re-renders skip this entirely. Reading the panels: pick the smallest K region where held-out likelihood has flattened and residual dispersion has stopped falling, then use the frontier plot to choose among candidates; if the frontier is still improving at $K = 80$, widen `tx$K_grid` rather than extrapolating.

- **Held-out likelihood:** log probability of the held-out words; higher is better, the most direct predictive criterion.
- **Semantic coherence:** for each topic's top words, how often they co-occur in the same documents, $\sum_{i < j} \log\{(D(v_i, v_j) + 1)/D(v_i)\}$, where $v_1, \dots, v_M$ are the topic's top words in de-

scending probability, $D(v_i)$ is the number of documents containing v_i , and $D(v_i, v_j)$ the number containing both; the denominator is the higher-ranked word v_i . Higher (less negative) means topics whose words travel together.

- **Exclusivity**: how much a topic's top words belong to it alone. Coherence can be gamed by a few common words, so read the coherence-exclusivity frontier, not either alone.
- **Residuals**: multinomial dispersion of the residuals; values above 1 signal more topics (or more structure) are needed.

```
if (tx$run_searchK) {
  heldout <- cache_rds(p_out("heldout_split.rds"), {
    set.seed(tx$seed)
    stm::make.heldout(stm_input$documents, stm_input$vocab,
                      proportion = tx$heldout_prop, seed = tx$seed)
  })

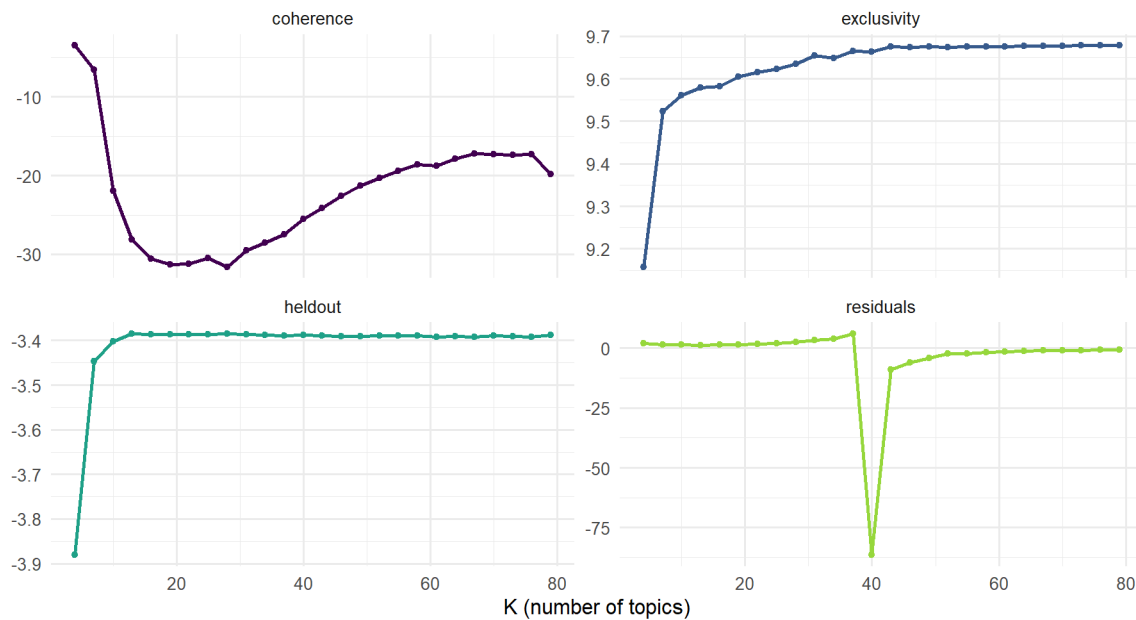
  searchK_tab <- cache_parquet(p_out("searchK_metrics.parquet"), {
    plan(multisession, workers = tx$workers)
    on.exit(plan(sequential), add = TRUE)
    future_map_dfr(tx$K_grid, function(K) {
      fit <- stm::stm(heldout$documents, heldout$vocab, K = K,
                     prevalence = ~ s(Year, df = tx$spline_df),
                     data = stm_input$meta, init.type = "Spectral",
                     seed = tx$seed, verbose = FALSE)

      # All metrics are computed into plain locals BEFORE tibble(): a tibble
      # column named `heldout` would shadow the env object `heldout` for every
      # later argument in the same call (data masking), turning
      # heldout$documents into $ on an atomic column.
      ho <- stm::eval.heldout(fit, heldout$missing)$expected.heldout
      rd <- stm::checkResiduals(fit, heldout$documents)$dispersion
      sc <- mean(stm::semanticCoherence(fit, heldout$documents))
      ex <- mean(stm::exclusivity(fit))
      tibble(K = K, heldout = ho, residuals = rd, coherence = sc, exclusivity =
ex)
    }, .options = furrr_options(seed = TRUE, packages = c("stm", "tibble")))
  })

  searchK_tab |>
    pivot_longer(~K, names_to = "metric", values_to = "value") |>
    ggplot(aes(K, value, color = metric)) +
    geom_line(linewidth = 1) + geom_point(size = 1.4) +
    scale_color_viridis_d(end = 0.85, guide = "none") +
    facet_wrap(~metric, scales = "free_y") +
    labs(x = "K (number of topics)", y = NULL,
         title = "searchK diagnostics over K = 4..80 by 3",
         subtitle = "Pick K at the coherence-exclusivity frontier where held-
out flattens; then set tx$K")
}
```

searchK diagnostics over K = 4..80 by 3

Pick K at the coherence-exclusivity frontier where held-out flattens; then set tx\$K



What it does: plots each K's semantic coherence against its exclusivity. Why: the two metrics trade off (a few common words can buy coherence at the cost of exclusivity), so K is chosen at the frontier, up and to the right, among candidates the four-panel diagnostics already screened; a K dominated on both axes is never the right choice.

```
if (tx$run_searchK) {
  searchK_tab |>
    ggplot(aes(coherence, exclusivity, color = K)) +
    geom_point(size = 2.5) +
    geom_text(aes(label = K), vjust = -0.8, size = 3, show.legend = FALSE) +
    scale_color_viridis_c() +
    labs(title = "Coherence vs exclusivity by K")
}
```

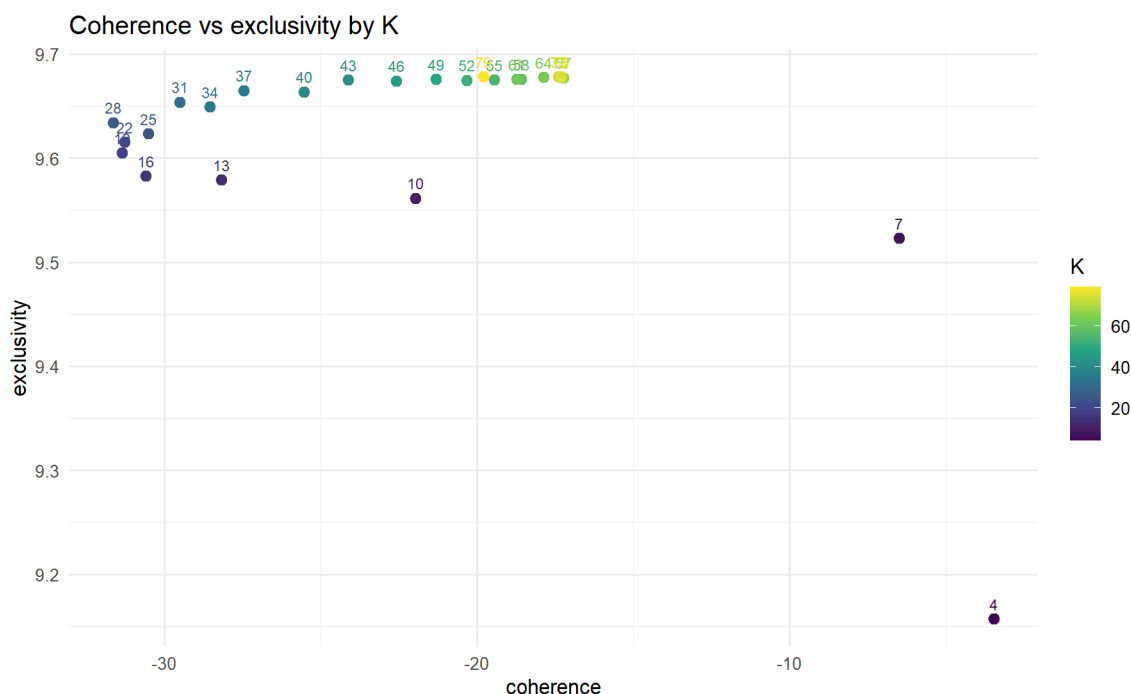


Figure 6: Coherence-exclusivity frontier: up and to the right dominates.

Set `tx$K` from these diagnostics and re-render (searchK reloads from its checkpoint, so the re-render is cheap). The simulation placeholder is `K = 15`.

Final STM

What it does: fits the final structural topic model at the chosen `tx$K`, with topical prevalence regressed on a smooth of Year (`df = 5`) and spectral initialization. Why the spline: attention to topics drifts over 2000-2025, and putting Year in the prevalence model estimates that drift instead of averaging it away; why spectral: it is deterministic given the data, removing initialization luck from a fit the whole downstream pipeline depends on. Checkpointed: re-renders reload the fit from `outputs/stm_fit.rds`.

```
stm_fit <- cache_rds(p_out("stm_fit.rds"), {
  stm::stm(stm_input$documents, stm_input$vocab, K = tx$K,
    prevalence = ~ s(Year, df = tx$spline_df),
    data = stm_input$meta, init.type = "Spectral",
    seed = tx$seed, verbose = FALSE)
})
```

Topic ranking by expected prevalence

A topic's expected corpus prevalence is the mean of its θ column over paragraphs: the share of corpus attention it commands. This is the ranking used to shortlist topics for the stance work in Script 2.

```

theta <- stm_fit$theta
colnames(theta) <- str_c("topic_", seq_len(tx$K))

# labelTopics output goes into `lbl`, NOT `frex`: the tibble below defines a
# column named frex, which would shadow an env object of the same name for the
# prob column (the same data-masking trap fixed in the searchK chunk).
lbl <- stm::labelTopics(stm_fit, n = 7)
topic_summary <- cache_parquet(p_out("topic_summary.parquet"), {
  tibble(topic      = seq_len(tx$K),
         prevalence = colMeans(theta),
         frex       = map_chr(seq_len(tx$K), ~ str_c(lbl$frex[.x, ], collapse =
", ")),
         prob       = map_chr(seq_len(tx$K), ~ str_c(lbl$prob[.x, ], collapse =
", "))) |>
    arrange(desc(prevalence))
})

topic_summary |>
  mutate(label = fct_reorder(str_c("T", topic, ": ", str_trunc(frex, 35)),
prevalence)) |>
  ggplot(aes(prevalence, label, fill = prevalence)) +
  geom_col(show.legend = FALSE) +
  scale_fill_viridis_c() +
  labs(x = "expected prevalence", y = NULL,
       title = "Topic ranking (name these, then set tx$selected_topics)")

```

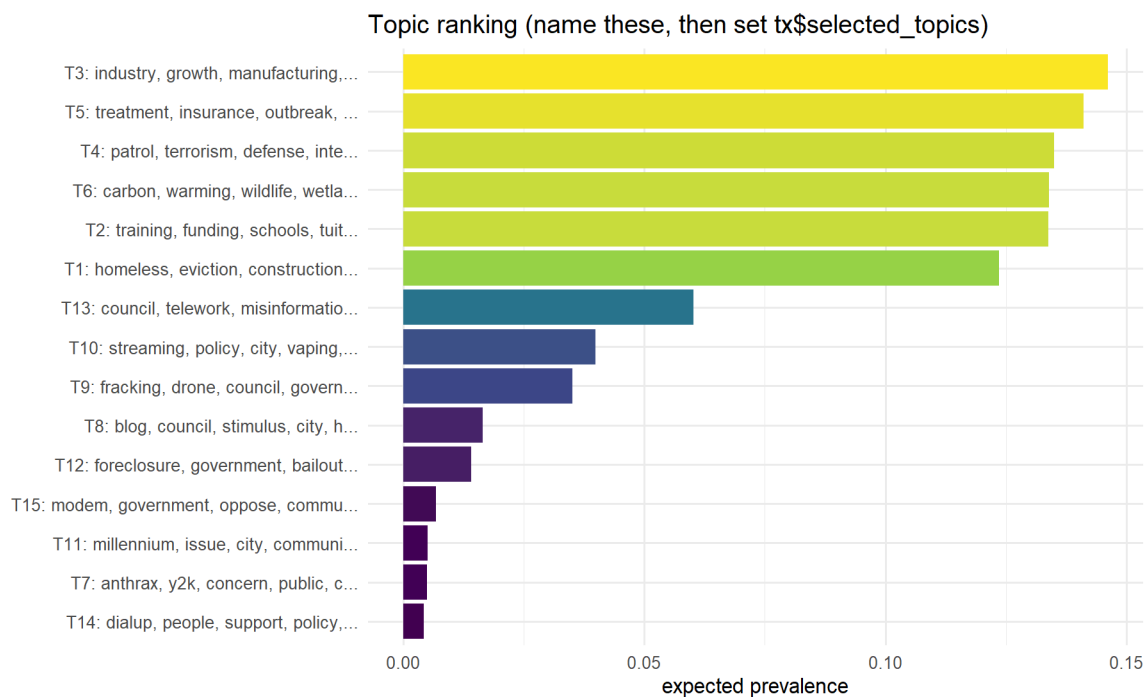


Figure 7: Topics ranked by expected corpus prevalence (mean theta).

What it does: the naming sheet as a table, one row per topic with prevalence, FREX, and highest-probability words. Why FREX for naming: frequent-only words blur topics together, exclusive-only words are rare oddities; FREX balances the two, which is what a human needs to tell neighboring topics apart.

```
topic_summary |>
  mutate(prevalence = round(prevalence, 3)) |>
  knitr::kable(caption = "Topics ranked by prevalence, with FREX (frequent AND
exclusive) and highest-probability words. FREX words are the naming aid: they
balance a word's frequency within the topic against its exclusivity to it.")
```

topic	prevalence	frex	prob
3	0.146	industry, growth, manufacturing, wages, trade, exports, jobs	industry, growth, manufacturing, wages, trade, exports, jobs
5	0.141	treatment, insurance, outbreak, epidemic, pandemic, screening, clinic	treatment, insurance, outbreak, epidemic, pandemic, screening, clinic
4	0.135	patrol, terrorism, defense, intelligence, surveillance, safety, border	patrol, terrorism, defense, intelligence, surveillance, safety, border
6	0.134	carbon, warming, wildlife, wetlands, renewable, conservation, climate	carbon, warming, wildlife, wetlands, renewable, conservation, climate
2	0.134	training, funding, schools, tuition, universities, curriculum, classrooms	training, funding, schools, tuition, universities, curriculum, classrooms
1	0.124	homeless, eviction, construction, gentrification, landlords, tenants, vacancy	homeless, eviction, construction, gentrification, landlords, tenants, vacancy
13	0.060	council, telework, misinformation, policy, quarantine, government, support	council, government, policy, support, issue, community, telework
10	0.040	streaming, policy, city, vaping, opioid, government, public	government, city, policy, council, streaming, community, support
9	0.035	fracking, drone, council, government, broadband, community, hashtag	council, government, fracking, drone, community, support, city
8	0.017	blog, council, stimulus, city, hurricane, support, public	council, blog, city, stimulus, hurricane, support, public

topic	prevalence	frex	prob
12	0.014	foreclosure, government, bailout, concern, community, policy, people	government, foreclosure, concern, bailout, community, policy, people
15	0.007	modern, government, oppose, community, concern, council, support	modern, government, oppose, community, council, concern, support
11	0.005	millennium, issue, city, community, people, council, government	millennium, issue, city, community, people, council, government
7	0.005	anthrax, y2k, concern, public, council, policy, support	anthrax, y2k, concern, public, council, policy, support
14	0.004	dialup, people, support, policy, public, government, council	dialup, people, support, policy, public, government, council

Table 2: Topics ranked by prevalence, with FREX (frequent AND exclusive) and highest-probability words. FREX words are the naming aid: they balance a word’s frequency within the topic against its exclusivity to it.

Representative paragraphs

`findThoughts` pulls the paragraphs with the highest θ per topic, the fastest way to sanity-check a topic before naming it.

```
top4 <- topic_summary$topic[1:4]
thoughts <- stm::findThoughts(stm_fit, texts = meta$para_uid, n = 3, topics = top4)
tibble(topic = rep(top4, each = 3),
        para_uid = unlist(thoughts$docs)) |>
  left_join(paragraphs |> dplyr::select(para_uid, text), by = "para_uid") |>
  mutate(text = str_trunc(text, 150)) |>
  knitr::kable(caption = "Three highest-theta paragraphs for the four most prevalent topics.")
```

topic	para_uid	text
3	A00757-04	the investment recession drone of exports less subsidies broadband exports market growth deficit jobs subsidies is industry must budget more recess...
3	A00624-05	market of budget exports market subsidies inflation market more manufacturing the deficit growth recession deficit budget trade taxes our trade tra...
3	A01083-02	budget for deficit deficit trade oppose investment that workers wages taxes investment deficit exports is taxes deficit in market subsidies workers...

topic	para_uid	text
5	A00942-07	prevention the prevention think vaccine care issue vaccine believe bailout screening that prevention for treatment less wellness more we screening ...
5	A00658-05	concern outbreak will nurses that nurses the nurses hospital hospital treatment care prevention treatment hospital insurance pandemic oppose patien...
5	A01066-06	hospital outbreak is more more disease not chatbot immunization mortality treatment not quarantine sanitation nurses pandemic to nurses hospital op...
4	A00216-03	less intelligence our surveillance safety patrol believe will a crime border border that believe protection crime police concern gangs patrol less ...
4	A00943-08	border border patrol quarantine border safety emergency believe that gangs terrorism and emergency enforcement weapons think should must weapons pa...
4	A00428-03	threat violence protection safety emergency terrorism less surveillance oppose oppose emergency protection defense patrol intelligence council safe...
6	A00940-07	our wetlands renewable flooding habitat wetlands and rivers rivers less emissions flooding chatbot solar rivers emissions conservation must climate...
6	A01040-02	cleanup renewable climate flooding very a climate public wildlife should solar flooding drought carbon pollution we will pesticides more rivers les...
6	A00516-06	wetlands rivers and need habitat wetlands in emissions we wetlands oppose less recycling public renewable recycling habitat conservation rivers cle...

Table 3: Three highest-theta paragraphs for the four most prevalent topics.

LLM topic-label drafting (optional, one call per topic)

Naming topics is analyst judgment; the LLM’s role here is strictly clerical: draft a candidate name and a one-sentence factual description from the same evidence the analyst reads (FREX words, highest-probability words, and the highest- θ paragraphs). Three properties keep this defensible:

- **One call per topic**, so the full run costs 15 requests, negligible against the API budget.
- **Drafts are frozen to disk and verified by hand.** The drafts land in `outputs/topic_labels_draft.parquet`; the analyst copies them to `outputs/`

topic_labels_verified.csv, edits the label column, and that verified file is the only naming artifact anything downstream reads.

- **Never fed back.** Labels are display strings; they enter no model, no prompt, and no estimate. The drafting model is also blind to Year and to any covariate, so it cannot leak design information into the names.

Adapting at work: set `tx$run_llm_labels = TRUE` and put the key in `.Renviron` (`OPENROUTER_API_KEY` now; the work endpoint's key and base URL later). The provider swap is the ONE config line `tx$llm_model`: any string not starting with "openrouter/" is routed by `make_label_chat()` to `ellmer::chat_openai(base_url = Sys.getenv("WORK_LLM_BASE_URL"))`, the OpenAI-compatible pattern for the work API. Structured output (`ellmer::type_object`) constrains the response to the two fields, so there is no free-text parsing step.

```
# One structured drafting call per topic. Wrapped in a function so pasting the
# chunk into a console keeps every intermediate local (no stale globals).
make_label_chat <- function(model, system_prompt) {
  if (startsWith(model, "openrouter/")) {
    ellmer::chat_openrouter(
      model      = sub("^openrouter/", "", model), # slug itself has a "/"
      system_prompt = system_prompt,
      params     = ellmer::params(temperature = 0),
      echo       = "none")
  } else {
    # WORK SWAP target: OpenAI-compatible endpoint; key + base URL from
    # .Renviron, never in code.
    ellmer::chat_openai(
      model      = model,
      base_url   = Sys.getenv("WORK_LLM_BASE_URL"),
      system_prompt = system_prompt,
      params     = ellmer::params(temperature = 0),
      echo       = "none")
  }
}

draft_topic_labels <- function(stm_fit, lbl, meta, paragraphs, K, model) {
  # Representative paragraphs for every topic, in dfm row order so findThoughts
  # indexes align with the fitted model.
  txt_in_dfm_order <- meta |>
    left_join(paragraphs |> dplyr::select(para_uid, text), by = "para_uid") |>
    pull(text)
  th <- stm::findThoughts(stm_fit, texts = txt_in_dfm_order, n = 3,
    topics = seq_len(K))

  prompts <- map_chr(seq_len(K), function(k) str_c(
    "Topic ", k, " of ", K, " from a topic model of public comments.\n",
    "FREX words (frequent AND exclusive to this topic): ",
```

```

str_c(lbl$frex[k, ], collapse = ", "), "\n",
"Highest-probability words: ",
str_c(lbl$prob[k, ], collapse = ", "), "\n",
"Representative paragraphs:\n",
str_c("- ", str_trunc(th$docs[[k]], 400), collapse = "\n")))

schema <- ellmer::type_object(
  label = ellmer::type_string(
    "Draft topic name, at most 4 words, noun phrase."),
  description = ellmer::type_string(
    "One factual sentence stating what these paragraphs discuss. Describe
only what the words and paragraphs show; no speculation about intent, stance,
or sentiment."))

chat <- make_label_chat(model, system_prompt = str_c(
  "You draft names for statistical topics so an analyst can verify them. ",
  "Base the name and description ONLY on the words and paragraphs provided."))

res <- ellmer::parallel_chat_structured(
  chat = chat, prompts = as.list(prompts), type = schema,
  max_active = 2, rpm = 30, on_error = "continue")

tibble(topic = seq_len(K)) |>
  bind_cols(as_tibble(res) |>
    dplyr::select(dplyr::any_of(c("label", "description"))))
}

if (isTRUE(tx$run_llm_labels)) {
  labels_draft <- cache_parquet(p_out("topic_labels_draft.parquet"), {
    draft_topic_labels(stm_fit, lbl, meta, paragraphs,
      K = tx$K, model = tx$llm_model)
  })
  labels_draft |>
    left_join(topic_summary |> dplyr::select(topic, prevalence), by = "topic")
|>
  arrange(desc(prevalence)) |>
  knitr::kable(caption = str_c(
    "DRAFT topic labels from ", tx$llm_model,
    ". Verify against the FREX sheet and representative paragraphs, then save
the edited table as outputs/topic_labels_verified.csv."))
} else {
  cat("LLM label drafting skipped (tx$run_llm_labels = FALSE). Name topics",
    "manually from the FREX sheet, or set the flag TRUE with the API key",
    "in .Renvirom for one drafting call per topic.\n")
}

```

LLM label drafting skipped (tx\$run_llm_labels = FALSE). Name topics manually from the FREX sheet, or set the flag TRUE with the API key in .Renviro for one drafting call per topic.

What it does: loads the analyst-verified label file when it exists and shows it; otherwise says so. Why a separate verified file: the LLM drafts are clerical suggestions, and the analyst's edited CSV is the only naming artifact Script 2 or any write-up may use, so verification is a file-existence contract rather than a promise.

```
# The verified file is the naming source of record. If your verified CSV kept
# the prevalence column, change the widths to [8, 22, 58, 12] to match. Create
# it by copying the
# draft parquet to CSV and editing:
#   arrow::read_parquet("outputs/topic_labels_draft.parquet") |>
#   readr::write_csv("outputs/topic_labels_verified.csv")
# then edit the label column by hand and keep only rows you have checked.
verified_path <- p_out("topic_labels_verified.csv")
if (file.exists(verified_path)) {
  topic_labels <- readr::read_csv(verified_path, show_col_types = FALSE)
  topic_labels |>
    knitr::kable(caption = "Verified topic labels (analyst-edited; the file
Script 2 and all write-ups should use).")
} else {
  cat("No outputs/topic_labels_verified.csv yet; topics remain numbered.",
      "Drafts, if generated, are NOT used downstream until verified.\n")
}
```

No outputs/topic_labels_verified.csv yet; topics remain numbered. Drafts, if generated, are NOT used downstream until verified.

Prevalence over time

estimateEffect regresses each topic's θ on the Year spline, giving the estimated attention trend with uncertainty from the model's approximate posterior.

```
# estimateEffect subsets its metadata by every variable name appearing in the
# formula, so tx$spline_df inside the formula becomes a lookup for a metadata
# column named "tx" and fails. Build the formula with literal numbers instead.
eff_formula <- as.formula(str_c("1:", tx$K, " ~ s(Year, df = ", tx$spline_df,
"))))
eff <- cache_rds(p_out("stm_year_effects.rds"), {
  stm::estimateEffect(eff_formula, stm_fit,
    metadata = stm_input$meta, uncertainty = "Global")
})
eff_tab <- map_dfr(top4, function(k) {
  pd <- plot(eff, covariate = "Year", topics = k, method = "continuous",
```

```

      printlegend = FALSE, omit.plot = TRUE) |> suppressWarnings()
tibble(topic = k, Year = pd$x,
      est = as.numeric(pd$means[[1]]),
      lo = pd$ci[[1]][1, ], hi = pd$ci[[1]][2, ])
})
eff_tab |>
  ggplot(aes(Year, est, color = factor(topic), fill = factor(topic))) +
  geom_ribbon(aes(ymin = lo, ymax = hi), alpha = 0.18, color = NA) +
  geom_line(linewidth = 1) +
  scale_color_viridis_d(end = 0.85, name = "topic") +
  scale_fill_viridis_d(end = 0.85, guide = "none") +
  labs(x = NULL, y = "expected prevalence", title = "Topic attention 2000-2025")

```

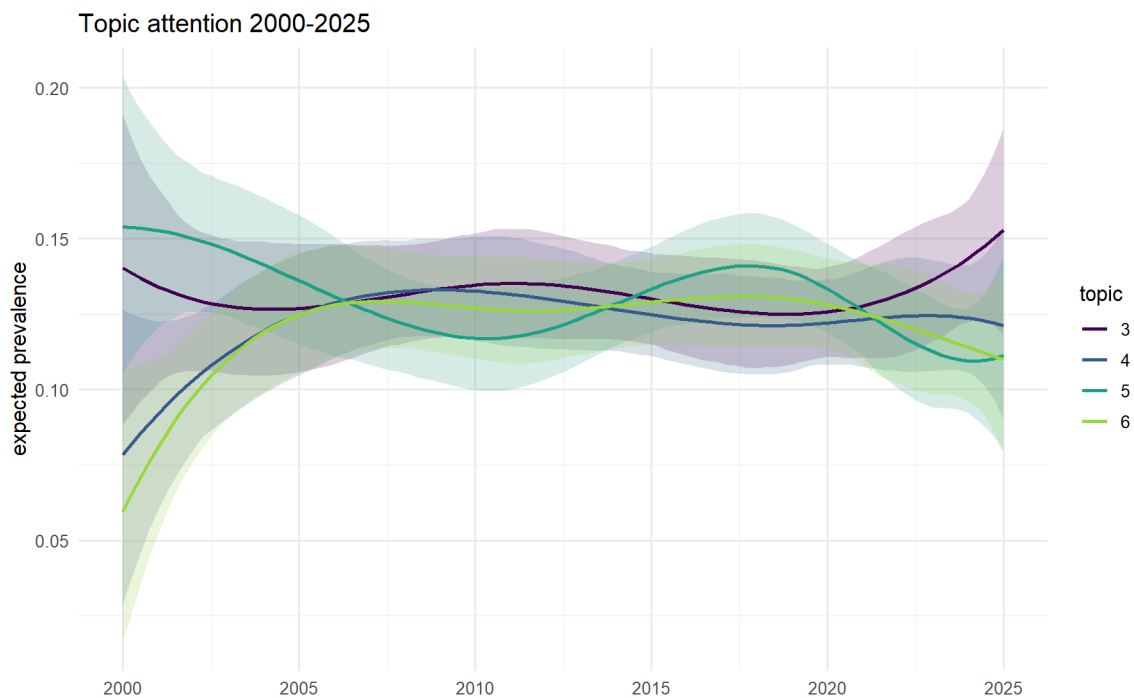


Figure 8: Estimated topical prevalence over time, top four topics.

Handoff to Script 2

What it does: assembles the paragraph-level table joined with the STM theta matrix and writes the full contract of files Script 2 reads. Why theta travels with the text: Script 2 assigns comments to topic strata from these paragraph shares and needs ids, Year, token counts, and text in one aligned table; nothing in Script 2 recomputes anything from this script.

```

theta_out <- cache_parquet(p_out("paragraph_theta.parquet"), {
  bind_cols(meta, as_tibble(theta)) |>
  left_join(paragraphs |> dplyr::select(para_uid, para_id, n_tokens, text),

```

```

        by = "para_uid")
  })
  cat("outputs/ written:\n",
    "- paragraphs.parquet          para_uid, atom_id, para_id, Year, n_tokens,
text\n",
    "- paragraph_theta.parquet      the above + topic_1..topic_K (STM theta)\n",
    "- topic_summary.parquet        topic, prevalence, frex, prob (naming sheet)\n",
    "- topic_labels_draft.parquet    LLM drafts (only if tx$run_llm_labels)\n",
    "- topic_labels_verified.csv     analyst-verified names (created by hand)\n",
    "    "- stm_fit.rds, lda_fit.rds, stm_year_effects.rds,
searchK_metrics.parquet\n")

```

```

outputs/ written:
- paragraphs.parquet          para_uid, atom_id, para_id, Year, n_tokens, text
- paragraph_theta.parquet      the above + topic_1..topic_K (STM theta)
- topic_summary.parquet        topic, prevalence, frex, prob (naming sheet)
- topic_labels_draft.parquet    LLM drafts (only if tx$run_llm_labels)
- topic_labels_verified.csv     analyst-verified names (created by hand)
- stm_fit.rds, lda_fit.rds, stm_year_effects.rds, searchK_metrics.parquet

```

Script 2 consumes paragraph_theta.parquet, topic_summary.parquet, and, when present, topic_labels_verified.csv. Before running it: name the topics (LLM drafts optional, verification mandatory), choose the ones for stance analysis, set tx\$selected_topics here, re-render, and write one stance proposition per selected topic there. Script 2's three parts are the LLM API setup (OpenRouter now, the work endpoint by the same one-line swap used above), the sampling design (stratified PPS of comments by token count within selected-topic strata, so the LLM sees a designed sample rather than the full corpus), and the stance measurement with design-weighted estimation.